
Adversarial Search - II

CS5491: Artificial Intelligence
ZHICHAO LU

Content Credits: Prof. Wei's CS4486 Course
and Prof. Boddeti's AI Course



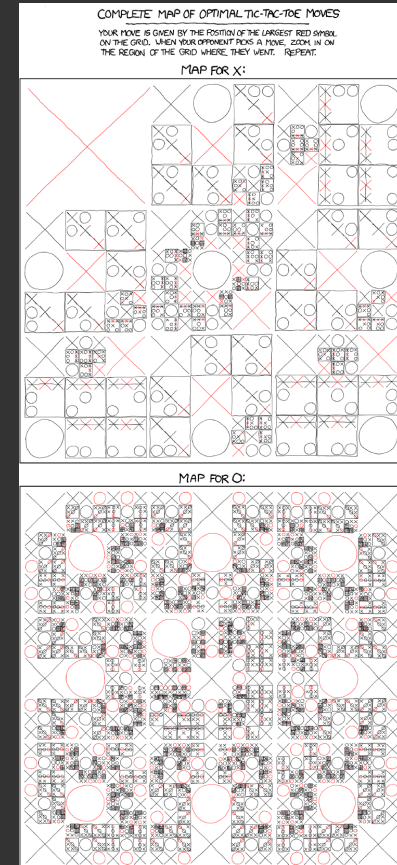
TODAY

Expectimax Games

Utilities

Reading

- Today's Lecture: RN Chapters 16.1-16.3

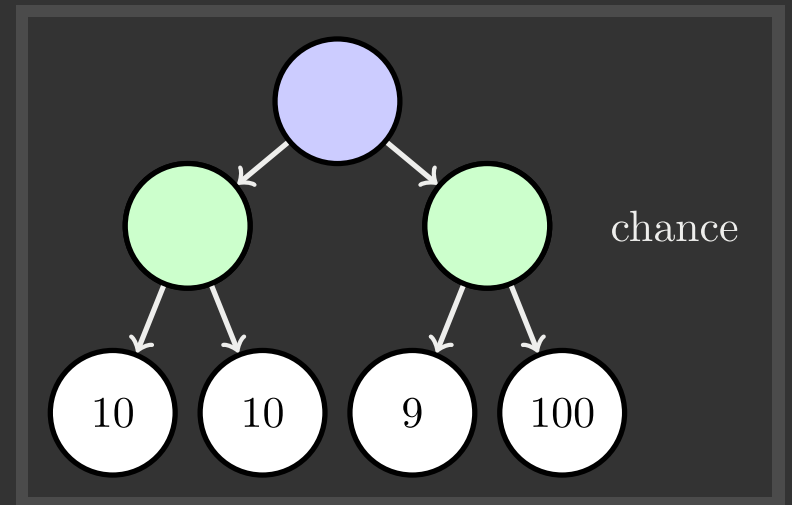
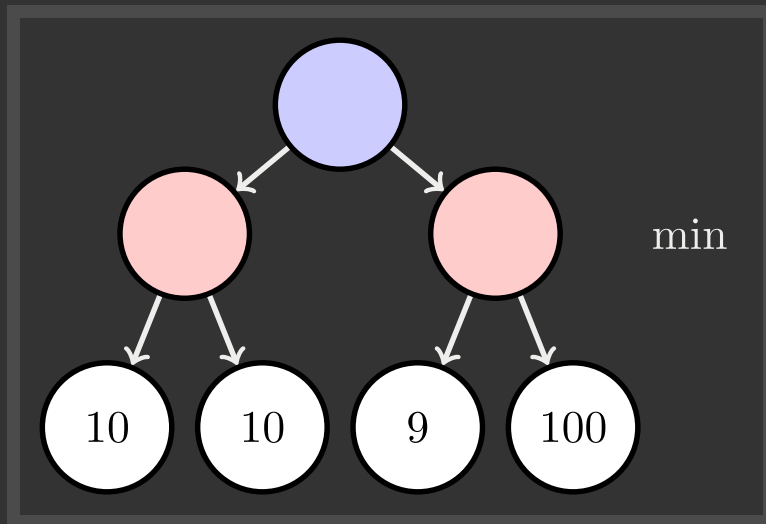


XKCD

UNCERTAIN OUTCOMES



WORST CASE VS AVERAGE CASE



Idea: Uncertain outcomes controlled by chance, not an adversary.

STOCHASTIC GAMES



STOCHASTIC GAMES

Why would we not know what the result of an action will be?

- Explicit randomness: rolling dice
- Unpredictable opponents: the ghosts respond randomly
- Actions can fail: when moving a robot, wheels might slip



STOCHASTIC GAMES

Why would we not know what the result of an action will be?

- Explicit randomness: rolling dice
- Unpredictable opponents: the ghosts respond randomly
- Actions can fail: when moving a robot, wheels might slip

Values should now reflect average-case (expectimax) outcomes, not worst-case (minimax) outcomes



EXPECTIMAX SEARCH

Expectimax search: compute the average score under optimal play



EXPECTIMAX SEARCH

Expectimax search: compute the average score under optimal play

- Max nodes as in minimax search



EXPECTIMAX SEARCH

Expectimax search: compute the average score under optimal play

- Max nodes as in minimax search
- Chance nodes are like min nodes but the outcome is uncertain



EXPECTIMAX SEARCH

Expectimax search: compute the average score under optimal play

- Max nodes as in minimax search
- Chance nodes are like min nodes but the outcome is uncertain
- Calculate their **expected utilities**
- i.e., take weighted average (expectation) of children



EXPECTIMAX SEARCH

Expectimax search: compute the average score under optimal play

- Max nodes as in minimax search
- Chance nodes are like min nodes but the outcome is uncertain
- Calculate their **expected utilities**
- i.e., take weighted average (expectation) of children

Later, we will learn how to formalize the underlying uncertain-result problems as **Markov Decision Processes**



EXPECTIMAX IMPLEMENTATION

Dispatch:

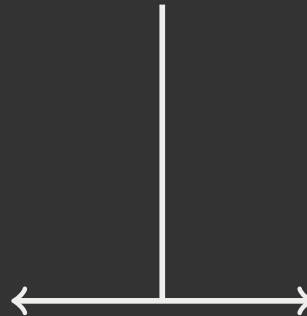
```
Function value(state)  
  if state is a terminal state then  
    | return the state's utility  
  end  
  if next agent is MAX then  
    | return max-value(state)  
  end  
  if next agent is EXP then  
    | return exp-value(state)  
  end
```

MAX Agent:

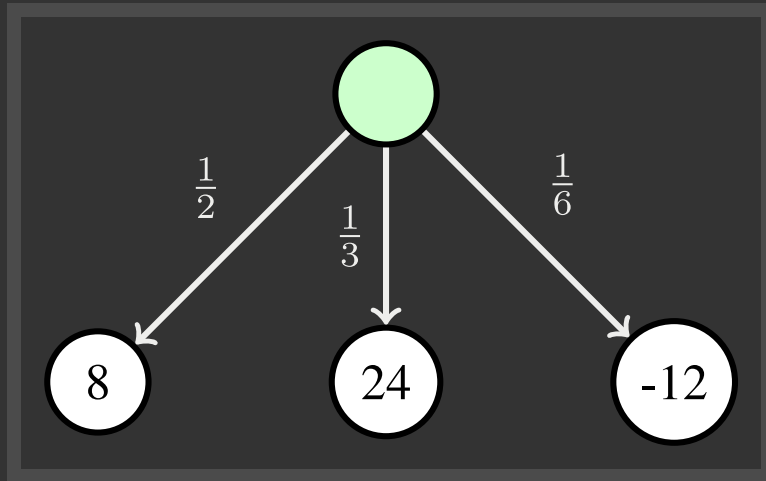
```
Function max-value(state)  
   $v \leftarrow -\infty$ ;  
  for each successor of state do  
    |  $v \leftarrow \max(v, \text{value}(\text{successor}))$ ;  
    | return  $v$ ;  
  end
```

EXP Agent:

```
Function exp-value(state)  
   $v \leftarrow 0$ ;  
  for each successor of state do  
    |  $p \leftarrow \text{probability}(\text{successor})$ ;  
    |  $v \leftarrow v + p \times \text{value}(\text{successor})$ ;  
    | return  $v$ ;  
  end
```



EXPECTIMAX IMPLEMENTATION



EXP Agent:

Function *exp-value*(state)

$v \leftarrow 0$;

for each successor of state **do**

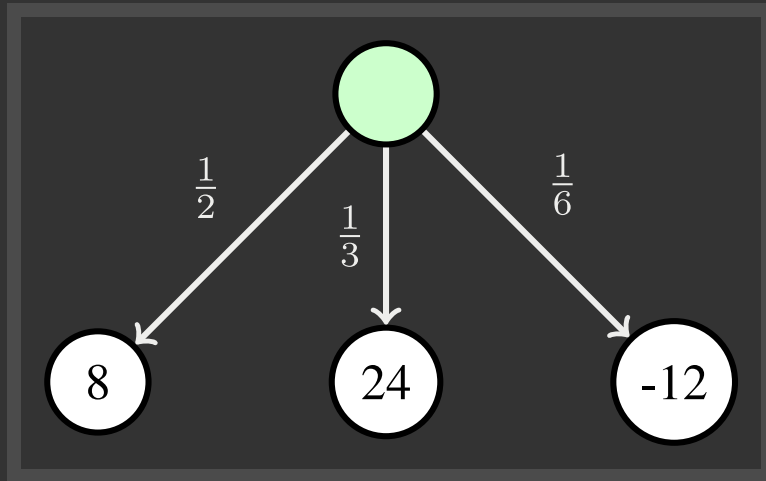
$p \leftarrow \text{probability}(\text{successor})$;

$v \leftarrow v + p \times \text{value}(\text{successor})$;

return v ;

end

EXPECTIMAX IMPLEMENTATION



EXP Agent:

Function *exp-value*(state)

$v \leftarrow 0$;

for each successor of state **do**

$p \leftarrow \text{probability}(\text{successor})$;

$v \leftarrow v + p \times \text{value}(\text{successor})$;

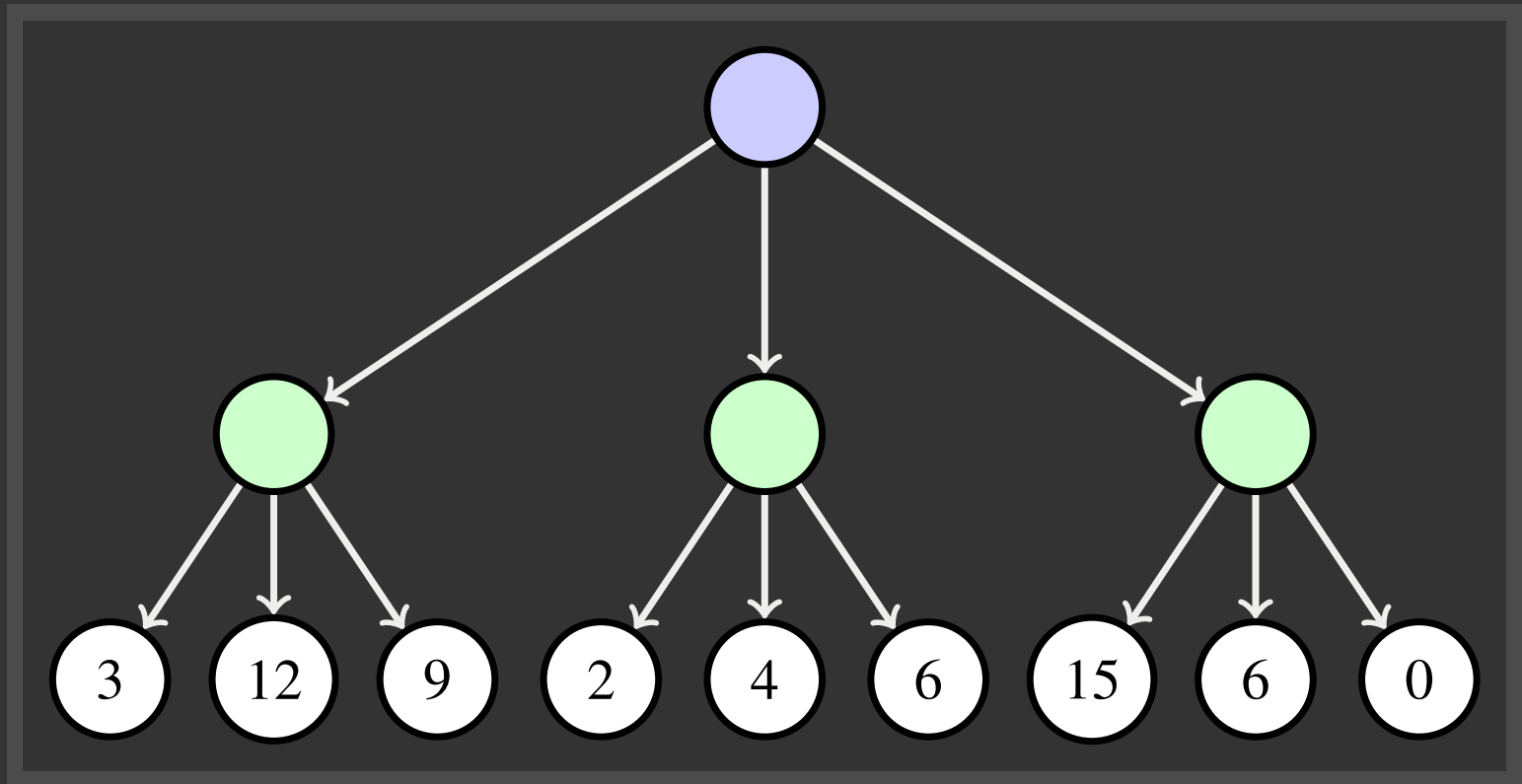
return v ;

end

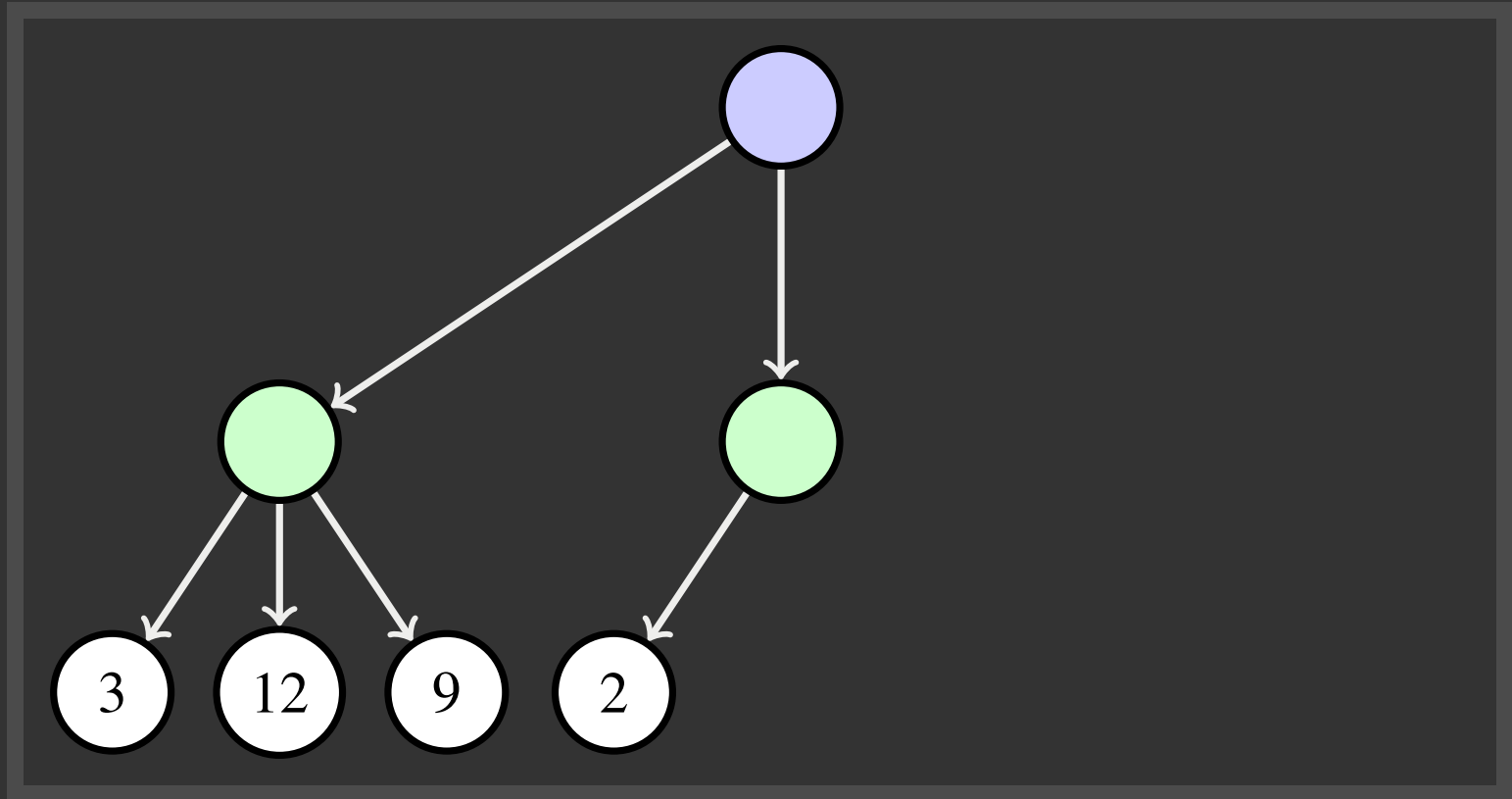
$$v = \frac{1}{2} * 8 + \frac{1}{3} * 24 - \frac{1}{6} * 12 = 10$$

(2)

EXPECTIMAX EXAMPLE



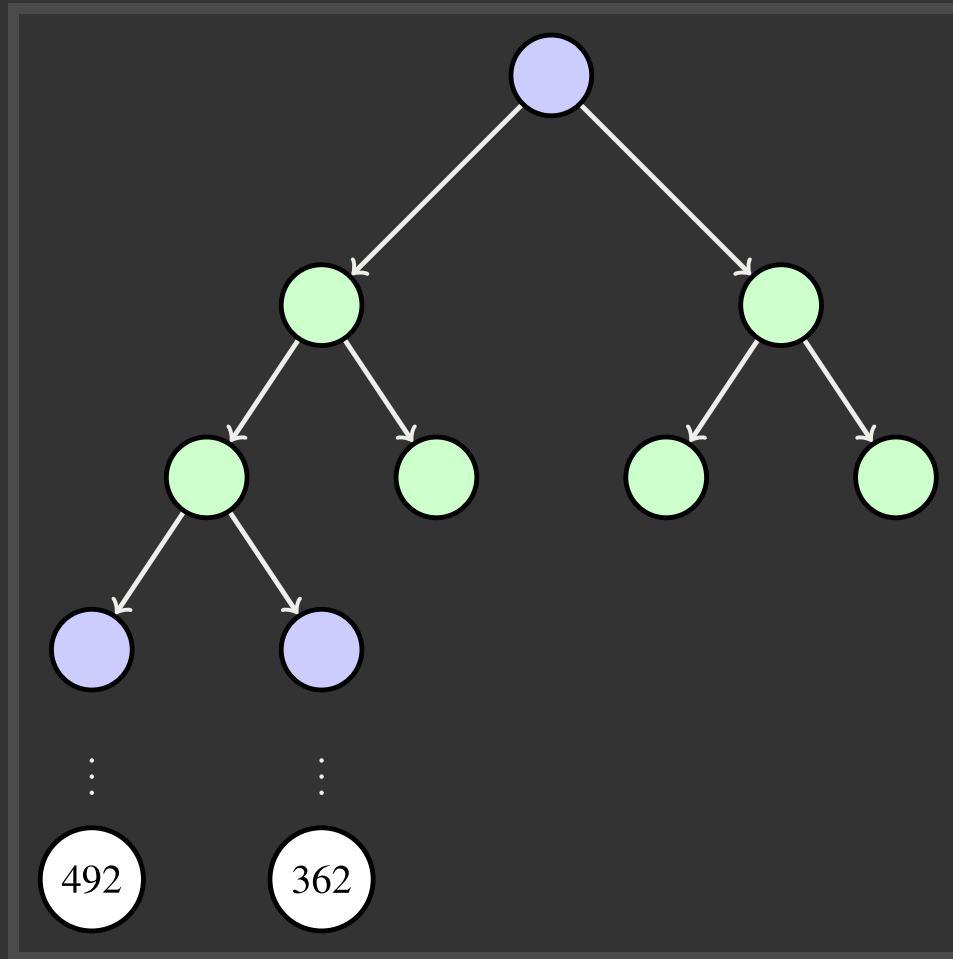
EXPECTIMAX PRUNING



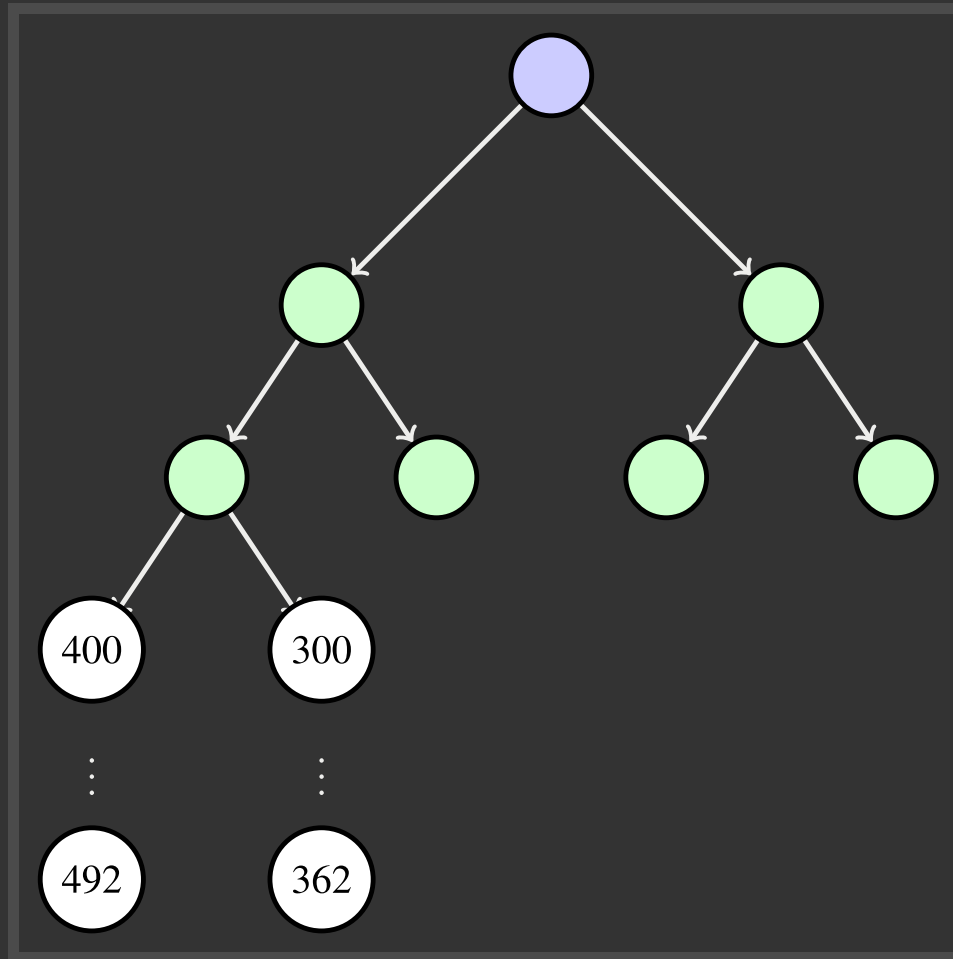
DEPTH-LIMITED EXPECTIMAX



DEPTH-LIMITED EXPECTIMAX



DEPTH-LIMITED EXPECTIMAX



PROBABILITIES



PRIMER: PROBABILITIES



PRIMER: PROBABILITIES

A **random variable** represents an event whose outcome is unknown.



PRIMER: PROBABILITIES

A **random variable** represents an event whose outcome is unknown.

A **probability distribution** is an assignment of weights to outcomes



PRIMER: PROBABILITIES

A **random variable** represents an event whose outcome is unknown.

A **probability distribution** is an assignment of weights to outcomes

Example: Traffic on freeway

- Random variable: $T =$ **whether there is traffic**
- Outcomes: $T \in \{none, mild, heavy\}$
- Distribution: $P(T = none) = 0.25$, $P(T = mild) = 0.50$,
 $P(T = heavy) = 0.25$



PRIMER: PROBABILITIES...



PRIMER: PROBABILITIES...

Some laws of probability (more later):

- Probabilities are always non-negative
- Probabilities over all possible outcomes sum to one



PRIMER: PROBABILITIES...

Some laws of probability (more later):

- Probabilities are always non-negative
- Probabilities over all possible outcomes sum to one

As we get more evidence, probabilities may change:

- $P(T = \textit{heavy}) = 0.25$, $P(T = \textit{heavy} \mid \textit{Hour} = 8am) = 0.60$
- We will talk about methods for reasoning and updating probabilities later



PRIMER: EXPECTATIONS

The expected value of a function of a random variable is the average, weighted by the probability distribution over outcomes



PRIMER: EXPECTATIONS

The expected value of a function of a random variable is the average, weighted by the probability distribution over outcomes

Example: How long to get to the airport?

	<i>None</i>	<i>mild</i>	<i>heavy</i>
Time:	20	30	60
Probability:	0.25	0.50	0.25



PRIMER: EXPECTATIONS

The expected value of a function of a random variable is the average, weighted by the probability distribution over outcomes

Example: How long to get to the airport?

	<i>None</i>		<i>mild</i>		<i>heavy</i>		
Time:	20		30		60		
	×	+	×	+	×	=	35
Probability:	0.25		0.50		0.25		

WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state



WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state

- Model could be a simple uniform distribution (roll a die)



WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state

- Model could be a simple uniform distribution (roll a die)
- Model could be sophisticated and require a great deal of computation



WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state

- Model could be a simple uniform distribution (roll a die)
- Model could be sophisticated and require a great deal of computation
- We have a chance node for any outcome out of our control: opponent or environment



WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state

- Model could be a simple uniform distribution (roll a die)
- Model could be sophisticated and require a great deal of computation
- We have a chance node for any outcome out of our control: opponent or environment
- The model might say that adversarial actions are likely.



WHAT PROBABILITIES TO USE?

In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state

- Model could be a simple uniform distribution (roll a die)
- Model could be sophisticated and require a great deal of computation
- We have a chance node for any outcome out of our control: opponent or environment
- The model might say that adversarial actions are likely.

For now, assume each chance node magically comes along with probabilities that specify the distribution over its outcomes



MODELING ASSUMPTIONS



THE DANGERS OF OPTIMISM AND PESSIMISM

Dangerous Optimism

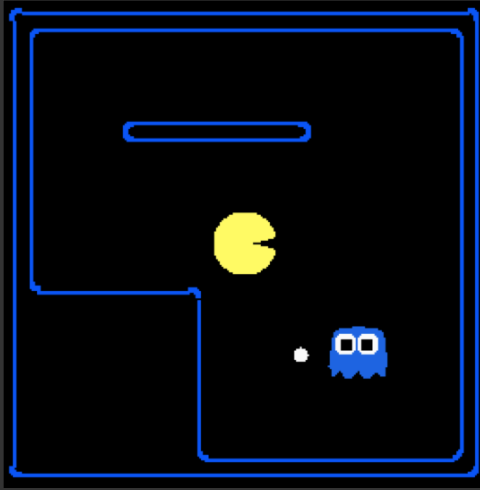
- Assuming chance when the world is adversarial

Dangerous Pessimism

- Assuming the worst case when it is not likely



ASSUMPTIONS VS REALITY



Pacman used depth 4 search with an eval function that avoids trouble

Ghost used depth 2 search with an eval function that seeks Pacman

Adversarial Ghost

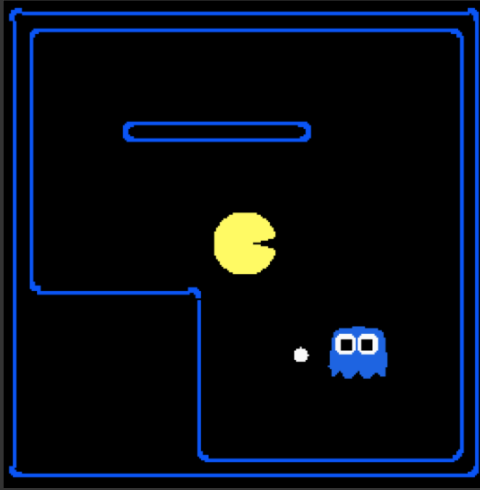
Random Ghost

Minimax Pacman

Expectimax Pacman



ASSUMPTIONS VS REALITY



Pacman used depth 4 search with an eval function that avoids trouble

Ghost used depth 2 search with an eval function that seeks Pacman

Adversarial Ghost

Random Ghost

Minimax Pacman

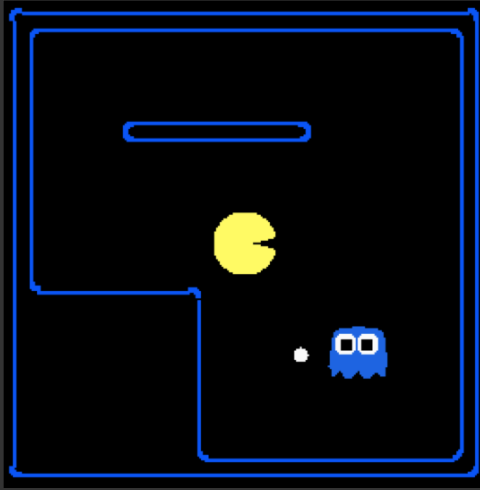
Won 5/5

Avg. Score: 483

Expectimax Pacman



ASSUMPTIONS VS REALITY



Pacman used depth 4 search with an eval function that avoids trouble

Ghost used depth 2 search with an eval function that seeks Pacman

Adversarial Ghost

Random Ghost

Minimax Pacman

Won 5/5

Won 5/5

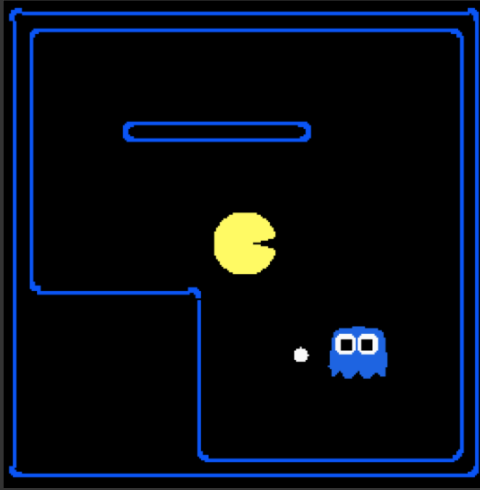
Avg. Score: 483

Avg. Score: 493

Expectimax Pacman



ASSUMPTIONS VS REALITY



Pacman used depth 4 search with an eval function that avoids trouble

Ghost used depth 2 search with an eval function that seeks Pacman

Adversarial Ghost

Random Ghost

Minimax Pacman

Won 5/5

Won 5/5

Avg. Score: 483

Avg. Score: 493

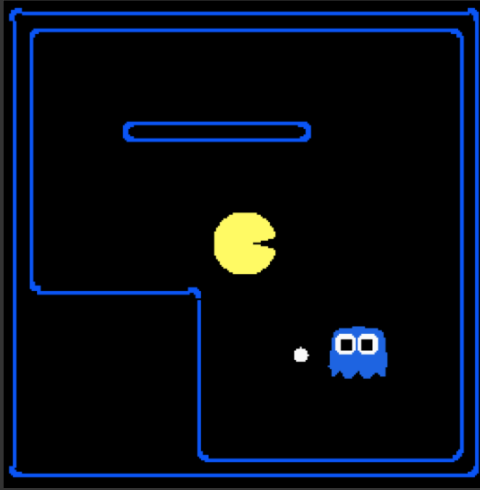
Expectimax Pacman

Won 1/5

Avg. Score: -303



ASSUMPTIONS VS REALITY



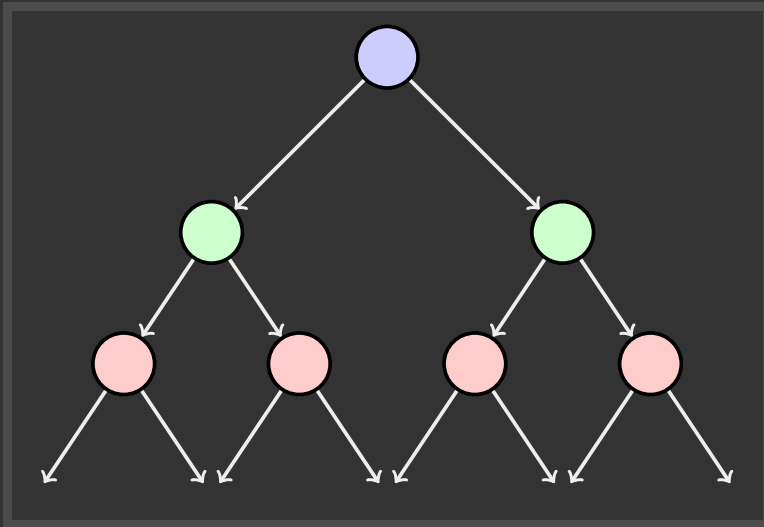
Pacman used depth 4 search with an eval function that avoids trouble

Ghost used depth 2 search with an eval function that seeks Pacman

	Adversarial Ghost	Random Ghost
Minimax Pacman	Won 5/5 Avg. Score: 483	Won 5/5 Avg. Score: 493
Expectimax Pacman	Won 1/5 Avg. Score: -303	Won 5/5 Avg. Score: 503

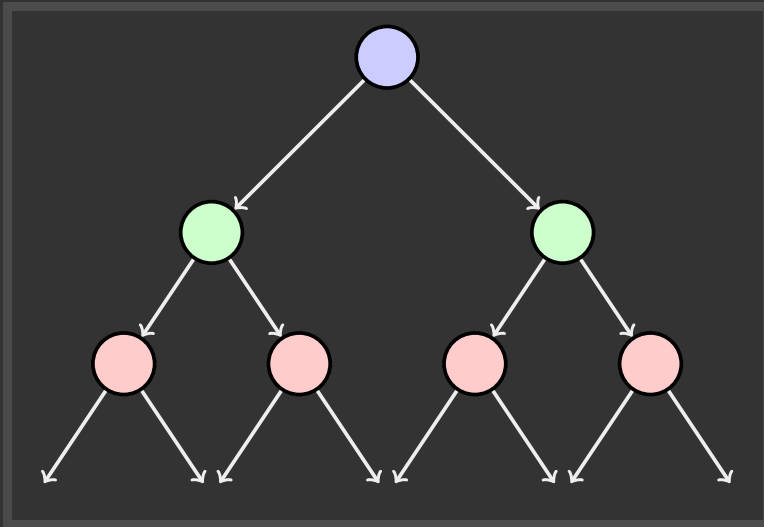


MIXED AGENT GAMES



E.g. Backgammon

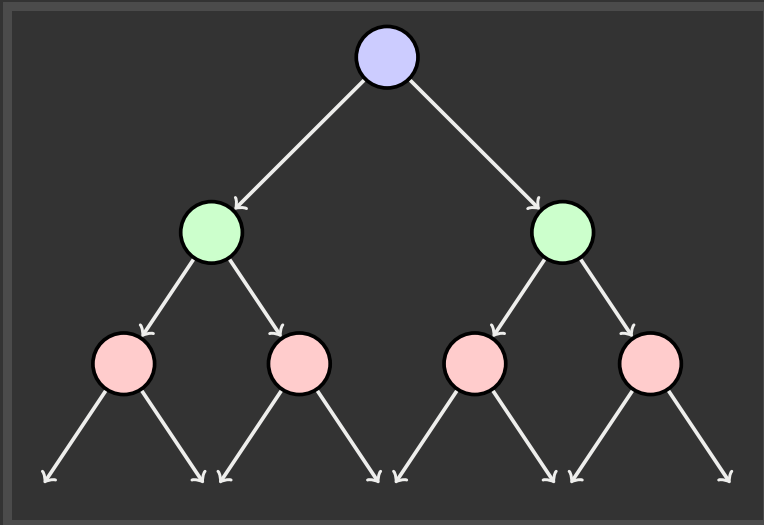
MIXED AGENT GAMES



E.g. Backgammon

Expectiminimax

MIXED AGENT GAMES

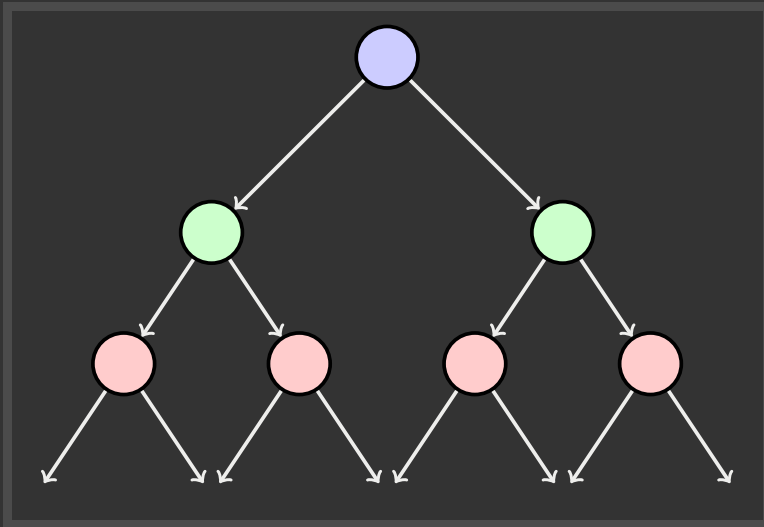


E.g. Backgammon

Expectiminimax

- Environment is an extra "random agent" player that moves after each min/max agent

MIXED AGENT GAMES



E.g. Backgammon

Expectiminimax

- Environment is an extra "random agent" player that moves after each min/max agent
- Each node computes the appropriate combination of its children

MIXED AGENT GAMES: BACKGAMMON



MIXED AGENT GAMES: BACKGAMMON

Dice rolls increase b: 21 possible rolls with 2 dice

- ≈ 20 legal moves
- Depth $2 = 20 \times (21 \times 20)^3 = 1.2 \times 10^9$



MIXED AGENT GAMES: BACKGAMMON

Dice rolls increase b: 21 possible rolls with 2 dice



- ≈ 20 legal moves
- Depth **2** = $20 \times (21 \times 20)^3 = 1.2 \times 10^9$

As depth increases, probability of reaching a given search node shrinks

- So usefulness of search is diminished
- So limiting depth is less damaging
- But pruning is trickier

MIXED AGENT GAMES: BACKGAMMON

Dice rolls increase b: 21 possible rolls with 2 dice



- ≈ 20 legal moves
- Depth **2** = $20 \times (21 \times 20)^3 = 1.2 \times 10^9$

As depth increases, probability of reaching a given search node shrinks

- So usefulness of search is diminished
- So limiting depth is less damaging
- But pruning is trickier

Historic AI: TDGammon uses depth-2 search + very good evaluation function + reinforcement learning: world-champion level play

MIXED AGENT GAMES: BACKGAMMON

Dice rolls increase b: 21 possible rolls with 2 dice



- ≈ 20 legal moves
- Depth **2** = $20 \times (21 \times 20)^3 = 1.2 \times 10^9$

As depth increases, probability of reaching a given search node shrinks

- So usefulness of search is diminished
- So limiting depth is less damaging
- But pruning is trickier

Historic AI: TDGammon uses depth-2 search + very good evaluation function + reinforcement learning: world-champion level play

UTILITIES



MAXIMUM EXPECTED UTILITY



MAXIMUM EXPECTED UTILITY

Why should we average utilities? Why not minimax?



MAXIMUM EXPECTED UTILITY

Why should we average utilities? Why not minimax?

Principle of maximum expected utility:

- A rational agent should choose the action that maximizes its expected utility, given its knowledge



MAXIMUM EXPECTED UTILITY

Why should we average utilities? Why not minimax?

Principle of maximum expected utility:

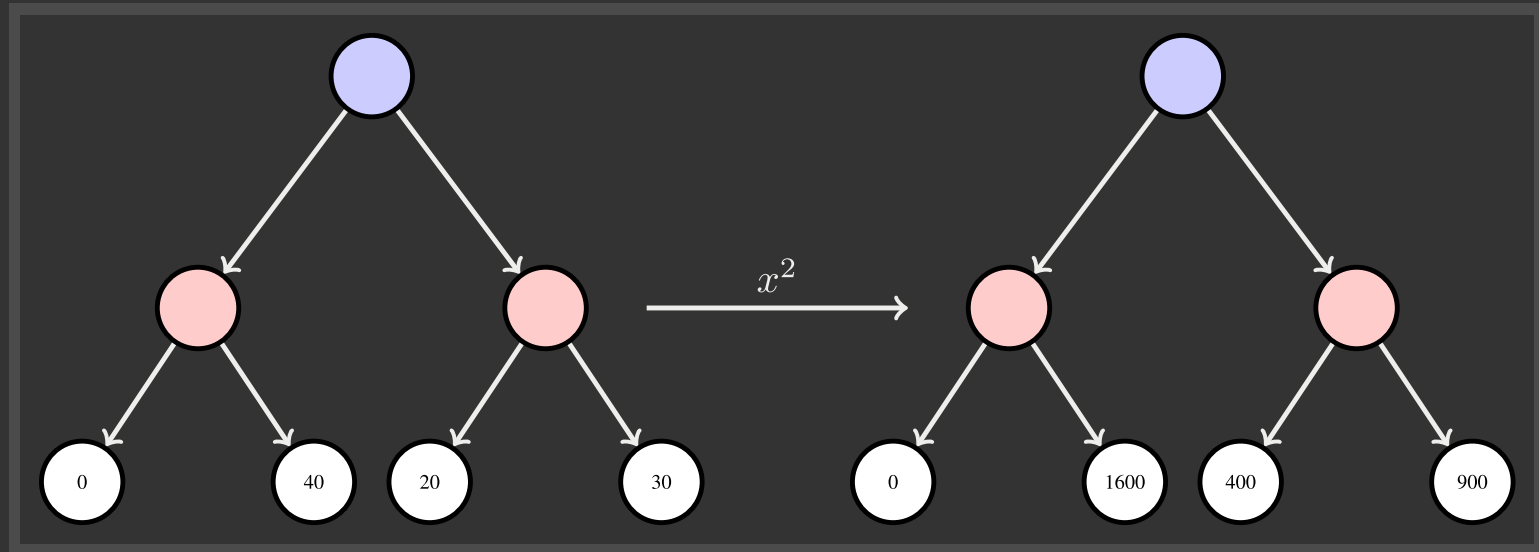
- A rational agent should choose the action that **maximizes its expected utility, given its knowledge**

Questions:

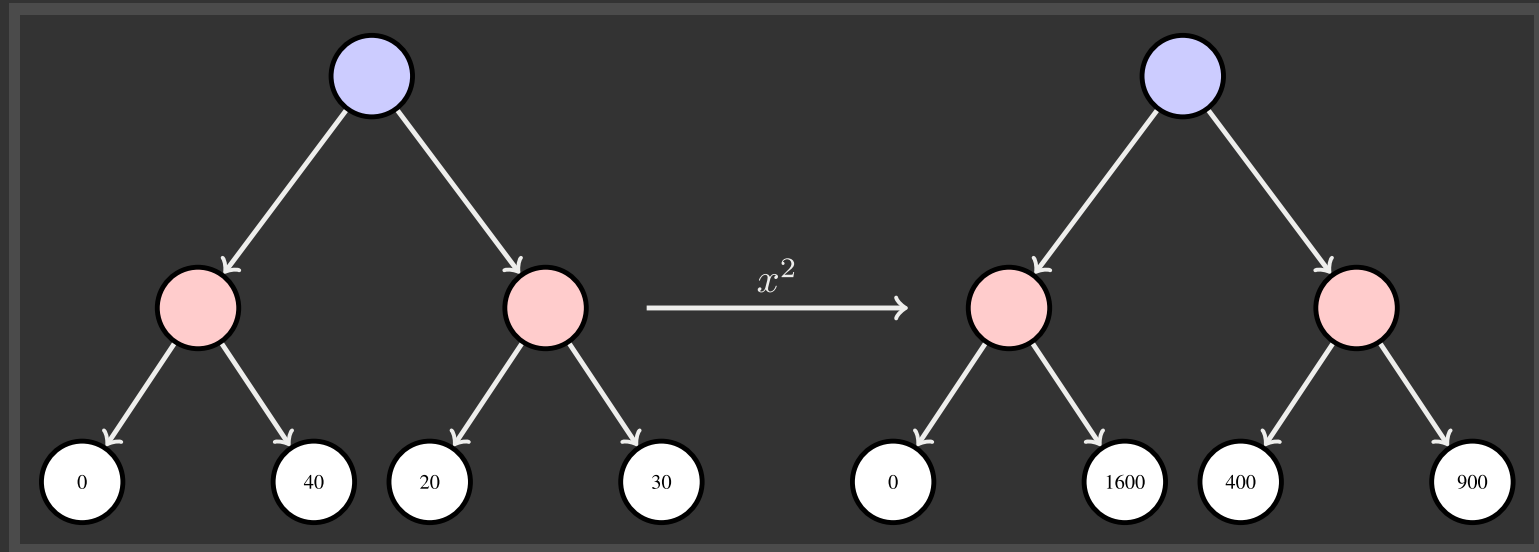
- Where do utilities come from?
- How do we know such utilities even exist?
- How do we know that averaging even makes sense?
- What if our behavior (preferences) cannot be described by utilities?



WHAT UTILITIES TO USE?

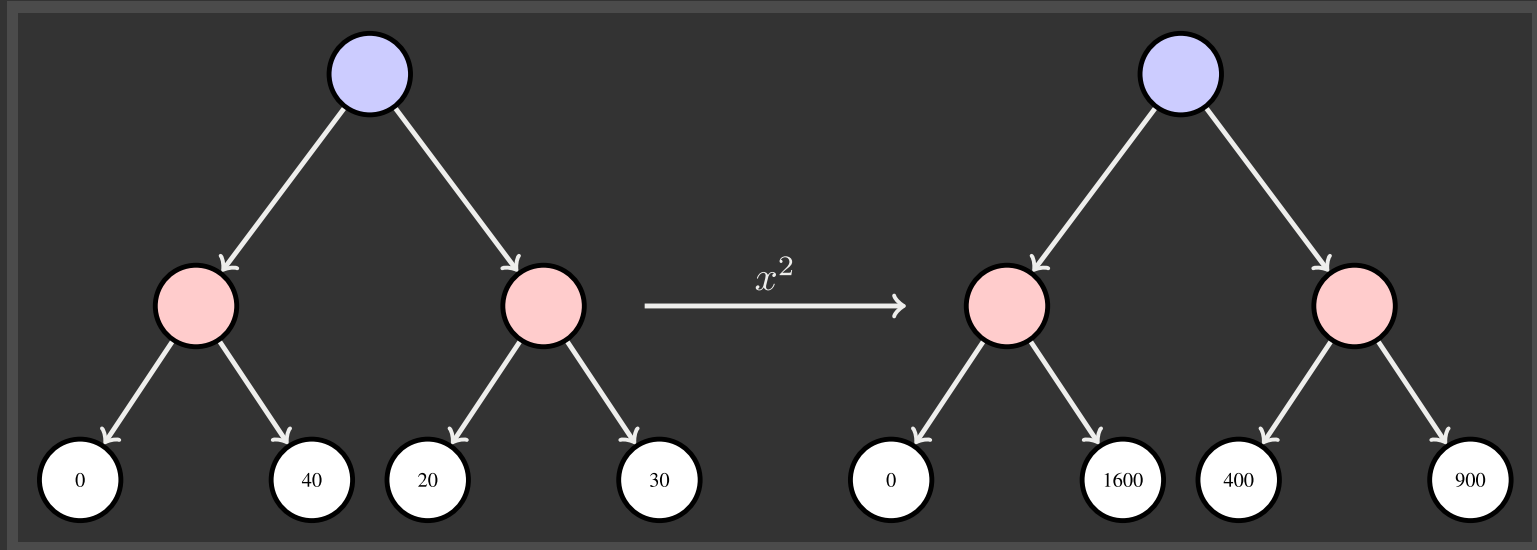


WHAT UTILITIES TO USE?



For worst-case minimax reasoning, terminal function scale does not matter

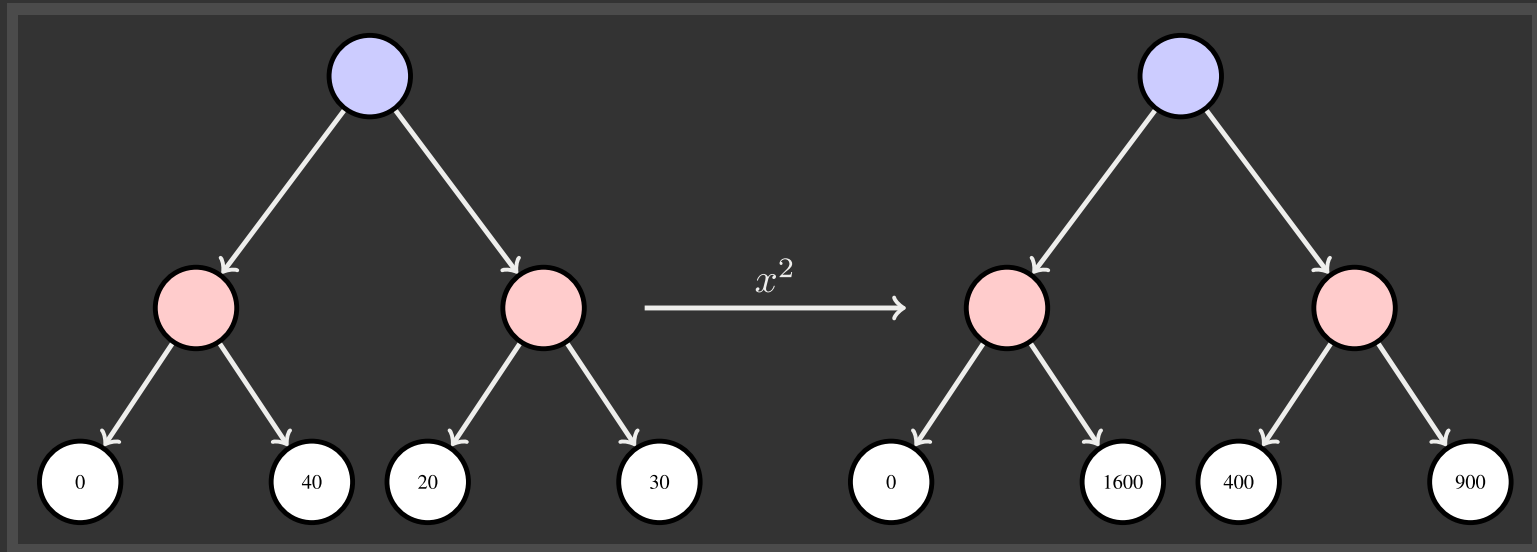
WHAT UTILITIES TO USE?



For worst-case minimax reasoning, terminal function scale does not matter

- We just want better states to have higher evaluations (get the ordering right)
- We call this **insensitivity to monotonic transformations**.

WHAT UTILITIES TO USE?



For worst-case minimax reasoning, terminal function scale does not matter

- We just want better states to have higher evaluations (get the ordering right)
- We call this **insensitivity to monotonic transformations**.

For average-case expectimax reasoning, we need magnitudes to be meaningful

UTILITIES

Utilities are functions from outcomes (states of the world) to real numbers that describe an agent's preferences



UTILITIES

Utilities are functions from outcomes (states of the world) to real numbers that describe an agent's preferences

Where do utilities come from?

- In a game, may be simple ($+1/ - 1$)
- Utilities summarize the agent's goals
- Theorem: any "rational" preferences can be summarized as a utility function



UTILITIES

Utilities are functions from outcomes (states of the world) to real numbers that describe an agent's preferences

Where do utilities come from?

- In a game, may be simple ($+1/ - 1$)
- Utilities summarize the agent's goals
- Theorem: any "rational" preferences can be summarized as a utility function

We hard-wire utilities and let behaviors emerge

- Why don't we let agents pick utilities?
- Why don't we prescribe behaviors?



Q & A



XKCD

